

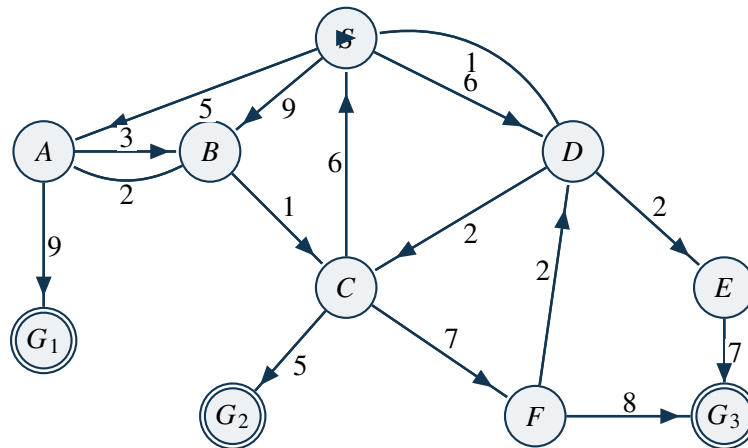
ESCOLA NAVAL

# Inteligência Artificial

## FORMULAÇÃO DE PROBLEMAS E PROCURA NÃO INFORMADA

### QUESTÃO 1 — PROCURA NUM GRAFO: BFS, DFS E UCS

Considere o grafo dirigido e ponderado seguinte. O estado inicial é  $S$  e os estados objetivo são  $G_1$ ,  $G_2$  e  $G_3$ . Só é permitido percorrer cada ligação no sentido da seta. Os números indicam os custos das ações.

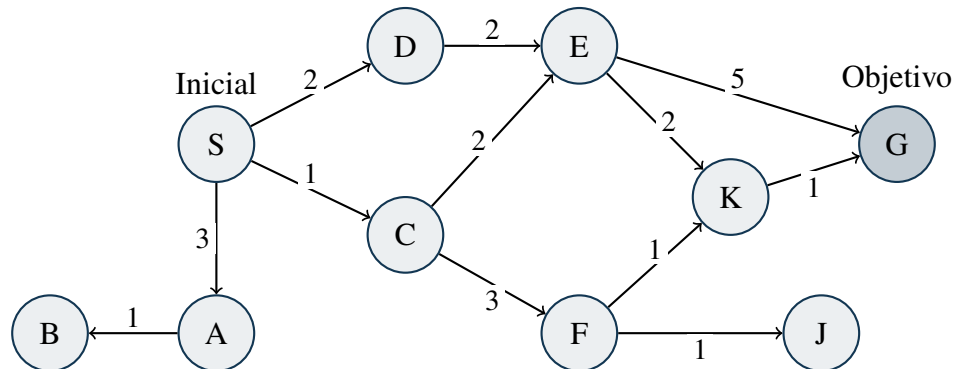


#### O que se pede

- Execute BFS, DFS e UCS até retirar os três objetivos. Para cada algoritmo, apresente a evolução de OPEN e VISITED, explicando as repetições ignoradas e as atualizações efetuadas.
- Desenhe a árvore de pesquisa no momento em que cada objetivo é retirado. Distinga os nós pendentes dos já visitados e assinale o caminho até ao objetivo desse momento.
- Indique a ordem de retirada dos estados e a ordem de confirmação dos objetivos. Para cada um de  $G_1$ ,  $G_2$  e  $G_3$ , reconstrua o caminho encontrado, a profundidade e o custo total.
- Compare os resultados. Distinga a ordem de retirada do caminho solução e explique por que motivo a UCS usa o custo acumulado, e não apenas o custo da última ligação.

## PROCURA NUM GRAFO: BFS, DFS E UCS

Considere o grafo orientado seguinte. O estado inicial é S e o estado objetivo é G. Os números nas ligações representam os custos das ações. Só é permitido percorrer cada ligação no sentido da seta.

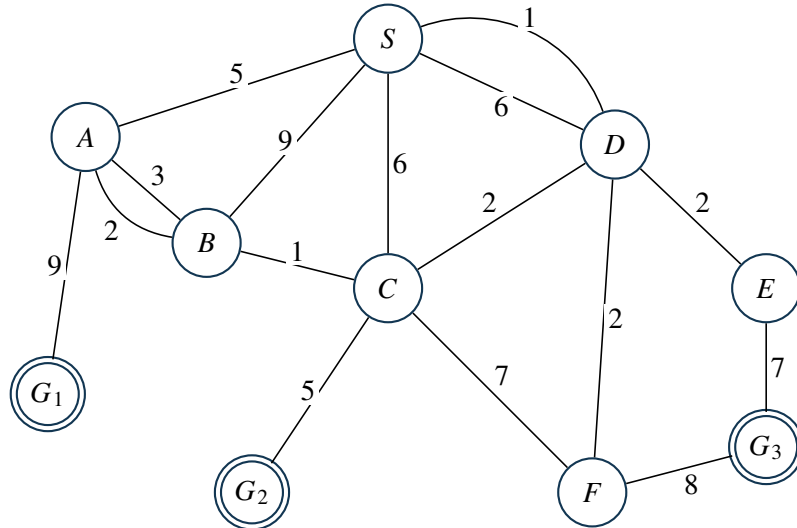


### O que se pede

- Execute BFS, DFS e UCS desde S até G. Para cada algoritmo, apresente a ordem de retirada dos nós e a evolução da fronteira.
- Reconstrua o caminho encontrado por cada algoritmo. Calcule a sua profundidade e o seu custo total.
- Compare os resultados: que algoritmo encontra o caminho de menor custo? Esse caminho tem também o menor número de ações?

## PROCURA NUM GRAFO: BFS, DFS E UCS

Considere o grafo dirigido e ponderado abaixo. O estado inicial é  $S$  e os estados objetivo são  $G_1$ ,  $G_2$  e  $G_3$ . Os números junto dos arcos indicam os respectivos custos. Pretende-se encontrar um caminho de  $S$  até qualquer objetivo.



### O que se pede

1. Aplique BFS, DFS e UCS, usando pesquisa em grafo e as convenções indicadas abaixo.
2. Para cada algoritmo, desenhe a árvore de pesquisa até à paragem e preencha, passo a passo, as estruturas OPEN/Fronteira e VISITED.
3. Indique a ordem de visita, a ordem de expansão, o objetivo encontrado, o caminho solução, a profundidade e o custo total.
4. Compare as soluções: qual tem menor profundidade e qual tem menor custo? Explique o papel de VISITED na prevenção de ciclos.

## ORDENAÇÃO POR TROCAS

Pretende-se ordenar a lista  $(2, 3, 1)$  por ordem crescente. A única ação permitida é trocar os elementos de duas posições diferentes. Pode trocar-se qualquer par de posições, mesmo que não sejam adjacentes. Cada troca custa 1.

### O que se pede

- a) Defina o estado inicial, a representação dos estados, as ações, a transição e o teste de objetivo.
- b) Enumere os estados possíveis e apresente os sucessores de cada estado para as trocas das posições  $(1, 2)$ ,  $(1, 3)$  e  $(2, 3)$ .
- c) Determine o número mínimo de trocas e apresente um caminho ótimo. Justifique por que motivo o BFS é adequado e por que é útil controlar os estados repetidos.

## MUNDO DO ASPIRADOR: BFS E DFS

Um aspirador desloca-se entre duas divisões, E (esquerda) e D (direita). Cada divisão está limpa ou suja. Pretende-se limpar ambas, independentemente da posição final do aspirador.

Representa-se um estado por  $(p, s_E, s_D)$ :  $p$  indica a posição e cada indicador de sujidade vale 1 se a divisão estiver suja e 0 se estiver limpa. Use os seguintes identificadores:

| Identificador | Estado      | Identificador | Estado      |
|---------------|-------------|---------------|-------------|
| 1             | $(E, 1, 1)$ | 5             | $(E, 1, 0)$ |
| 2             | $(D, 1, 1)$ | 6             | $(D, 1, 0)$ |
| 3             | $(E, 0, 1)$ | 7             | $(E, 0, 0)$ |
| 4             | $(D, 0, 1)$ | 8             | $(D, 0, 0)$ |

O estado inicial é 1. Existem três ações, geradas sempre pela ordem **A, D, E**: A aspira a divisão atual; D move para a direita; E move para a esquerda. Aspirar uma divisão limpa ou mover para a divisão onde já se encontra mantém o estado. Cada ação custa 1.

### Regras de execução

Use procura em grafo. Registe um estado como alcançado logo que é inserido na fronteira e não o insira novamente. O estado inicial é registado antes de começar.

No BFS, insira no fim e retire do início da fila. No DFS, insira e retire no topo da pilha, situado à direita. Como os sucessores são inseridos pela ordem A, D, E, o último sucessor novo inserido será o primeiro a sair no DFS.

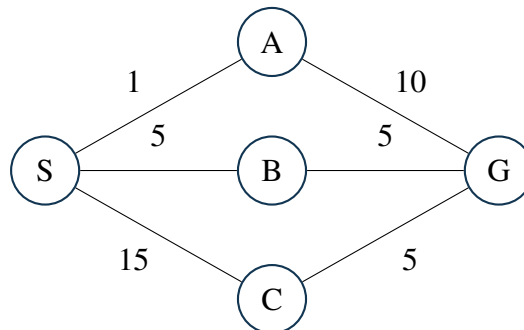
Teste o objetivo **depois de retirar** um nó. O objetivo é retirado, mas não expandido. Na apresentação da fronteira, use os identificadores numéricos dos estados.

### O que se pede

- Indique os estados objetivo e os sucessores do estado 1.
- Execute BFS e DFS até ao primeiro objetivo. Registe a fronteira após cada operação e reconstrua os caminhos através dos pais.
- Compare o custo dos caminhos, o número de nós retirados e o número de nós expandidos. O exemplo prova que uma estratégia é sempre melhor?

## PROCURA DE CUSTO UNIFORME

Considere o grafo não orientado seguinte. Cada ligação pode ser percorrida nos dois sentidos e o número junto à aresta indica o custo da ação. O estado inicial é S e o objetivo é G. Os custos são dados didáticos.



### Regras de execução

Use UCS com uma fronteira OPEN ordenada pelo menor custo acumulado  $g$ . Inicialmente,  $g(S) = 0$  e os restantes custos valem  $\infty$  (ainda não foi encontrado um caminho).

Gere os vizinhos por ordem alfabética. Para um sucessor  $m$  de  $n$ , calcule  $g(n) + c(n, m)$ . Se este valor for estritamente menor do que o melhor custo conhecido de  $m$ , atualize o custo, o pai e a entrada em OPEN. Não mantenha entradas duplicadas. Em igualdade de prioridades, use FIFO.

Teste o objetivo apenas quando o nó é retirado de OPEN.

### O que se pede

- Execute UCS, mostrando OPEN após cada expansão, os custos e as atualizações dos pais.
- Quando A gera G com custo 11, já foi encontrada uma solução. Por que motivo não deve o algoritmo terminar nessa altura?
- Reconstrua a solução final, verifique o custo e conte os nós retirados e expandidos. Que condições justificam a optimalidade neste exemplo?